

SUPMEAL

Documentation technique

Application de gestion de recettes et de planification de repas



Sommaire

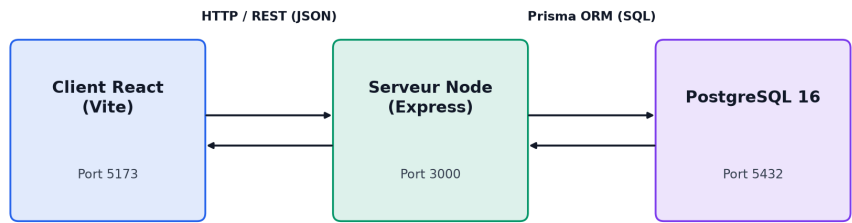
Sommaire	2
1. Présentation du projet	3
2. Architecture générale.....	3
3. Stack technique	3
Backend	3
Frontend	4
Infrastructure	4
4. Prérequis et variables d'environnement	4
Prérequis	4
Variables d'environnement	4
5. Guide de déploiement	5
Déploiement local (développement).....	5
Comptes de test (créés automatiquement par le seed).....	5
7. Diagramme de classes	7
8. Diagramme de cas d'utilisation	8
9. Sécurité.....	9
10. API Documentation	9
Authentification.....	9
Recettes.....	9
Cookbooks	10
Recherche, planification, profils.....	10
Import / Export & invitations.....	11

1. Présentation du projet

SUPMEAL est une application web de gestion de recettes et de planification de repas. Elle permet aux utilisateurs de créer, organiser et partager des recettes, de planifier leurs repas hebdomadaires et de découvrir des recettes publiques partagées par la communauté.

2. Architecture générale

L'application suit une architecture trois tiers stricte :



Principe : le client n’a aucune partie de la logique métier. Il se contente d'envoyer des requêtes HTTP à l'API REST et d'afficher les réponses JSON. Toute la logique (authentification, permissions, validation) est côté serveur.

3. Stack technique

Backend

Technologie	Justification
Node.js	Environnement JavaScript côté serveur, excellente compatibilité avec l'écosystème npm et
Express	Framework web standard pour les API REST Node.js
Prisma	ORM avec migrations automatiques, génération de client, support PostgreSQL pour faciliter le processus
PostgreSQL	SGBD relationnel robuste, open-source, excellent support JSON et requêtes complexes
bcrypt	Hachage sécurisé des mots de passe avec salage automatique (10 rounds)
jsonwebtoken	Authentification stateless via JWT, évite la gestion de sessions côté serveur
Passport.js	Middleware d'authentification modulaire, simplifie l'intégration OAuth2
passport-google-oauth20	Stratégie OAuth2 Google pour Passport
Nodemailer	Envoi d'emails transactionnels (vérification, réinitialisation mdp) via SMTP
Multer	Gestion des uploads de fichiers (images de recettes), validation MIME et taille
swagger-jsdoc / swagger-ui-express	Documentation API interactive générée depuis les annotations JSDoc
csv-parse / csv-stringify	Import / export au format CSV

Frontend

Technologie	Justification
React	Bibliothèque UI déclarative, composants réutilisables, écosystème riche
Vite	Bundler ultra-rapide avec HMR (Hot Module Replacement) pour le développement
React Router	Routage côté client standard pour les SPA React
Tailwind CSS	Framework CSS pour un développement UI rapide et cohérent
Axios	Client HTTP avec intercepteurs (ajout automatique du token JWT sur chaque requête)

Infrastructure

Technologie	Justification
Docker & Docker Compose	Conteneurisation des 3 services (client, server, db), déploiement reproductible en une commande
node:20-alpine	Image Docker légère pour Node.js, réduit la taille des images
postgres:16-alpine	Image Docker officielle PostgreSQL, légère

4. Prérequis et variables d'environnement

Prérequis

- Docker Desktop
- Git

Variables d'environnement

Copiez `.env.example` en `.env` et remplissez les valeurs :

Obligatoires:

Variable	Description
POSTGRES_USER	Nom d'utilisateur PostgreSQL
POSTGRES_PASSWORD	Mot de passe PostgreSQL
POSTGRES_DB	Nom de la base de données
JWT_SECRET	Clé secrète JWT (min. 32 caractères) — générez avec <code>openssl rand -hex 32</code>
FRONTEND_URL	URL du frontend (ex : <code>http://localhost:5173</code>)

Optionnelles:

Variable	Description
GOOGLE_CLIENT_ID	Client ID Google OAuth2 (laisser "changeme" pour désactiver)
GOOGLE_CLIENT_SECRET	Client Secret Google OAuth2
GOOGLE_CALLBACK_URL	URL de callback OAuth2
SMTP_HOST	Hôte SMTP pour les emails (ex : <code>smtp.gmail.com</code>)
SMTP_PORT	Port SMTP (ex : 587)
SMTP_USER	Adresse email SMTP
SMTP_PASS	Mot de passe d'application SMTP

5. Guide de déploiement

Déploiement local (développement)

1. Cloner le dépôt

git clone <https://github.com/sdiene/SUPMEAL.git>

2. Configurer les variables d'environnement

Copier le fichier .env.example et remplacer les valeurs par les vôtres dans .env

3. Lancer l'application (build + migrations + seed + démarrage)

docker compose up --build

- Frontend : <http://localhost:5173>
- API : <http://localhost:3000>
- API Docs : <http://localhost:3000/docs>

Comptes de test (créés automatiquement apres le docker-compose up)

Nom	Email	Mot de passe	Rôle
Chacour Diene	chacour@gmail.com	CourCha1	OWNER du cookbook de démonstration
Lamine Diene	lamine@gmail.com	Lamine1	EDITOR du cookbook de démonstration

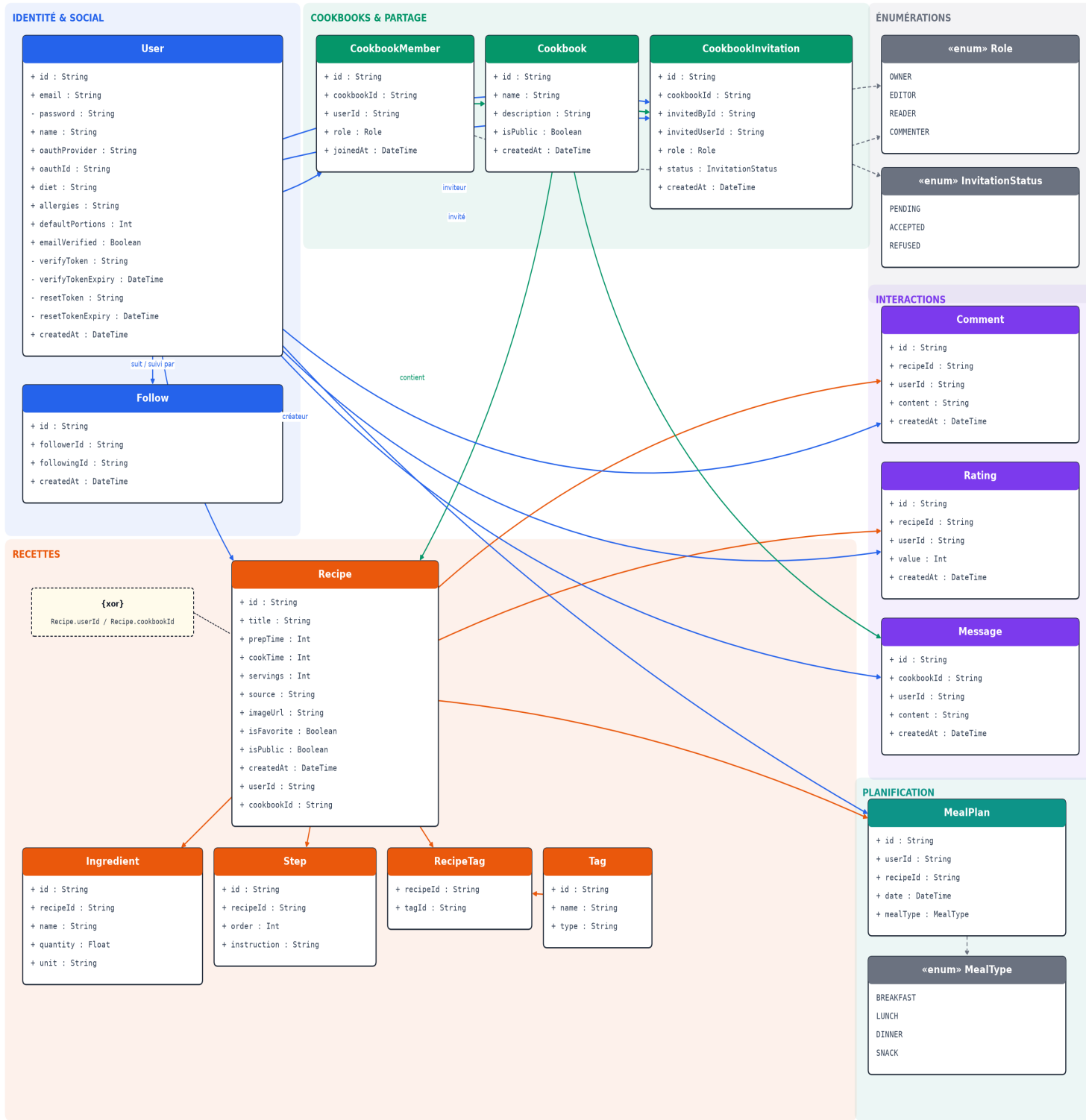
Notes : Ce seed a été mis en place pour tous problèmes liés à l'authentification lors du test

6. Structure du projet

```
.
├─ client/                                React (Vite) – interface utilisateur
│  ├─ src/
│  │  ├─ api/                            appels vers le backend
│  │  ├─ components/                    composants réutilisables
│  │  ├─ pages/                         une page par route de l'appli
│  │  ├─ context/                       état global (authentification...)
│  │  ├─ hooks/                         logique réutilisable (hooks React)
│  │  ├─ App.jsx
│  │  └─ main.jsx                       point d'entrée
│  └─ package.json
│
├─ server/                                Express + Prisma – API et logique métier
│  ├─ src/
│  │  ├─ routes/                        définition des endpoints
│  │  ├─ controllers/                  reçoit les requêtes, appelle les services
│  │  ├─ services/                     logique métier, accès aux données
│  │  ├─ middlewares/                 authentication, permissions...
│  │  └─ index.js                      point d'entrée du serveur
│  ├─ prisma/
│  │  └─ schema.prisma                 modèle de la base de données
│  └─ package.json
│
├─ docs/                                documentation technique + manuel utilisateur (PDF)
├─ docker-compose.yml                   lance client + serveur + base de données
└─ README.md
```

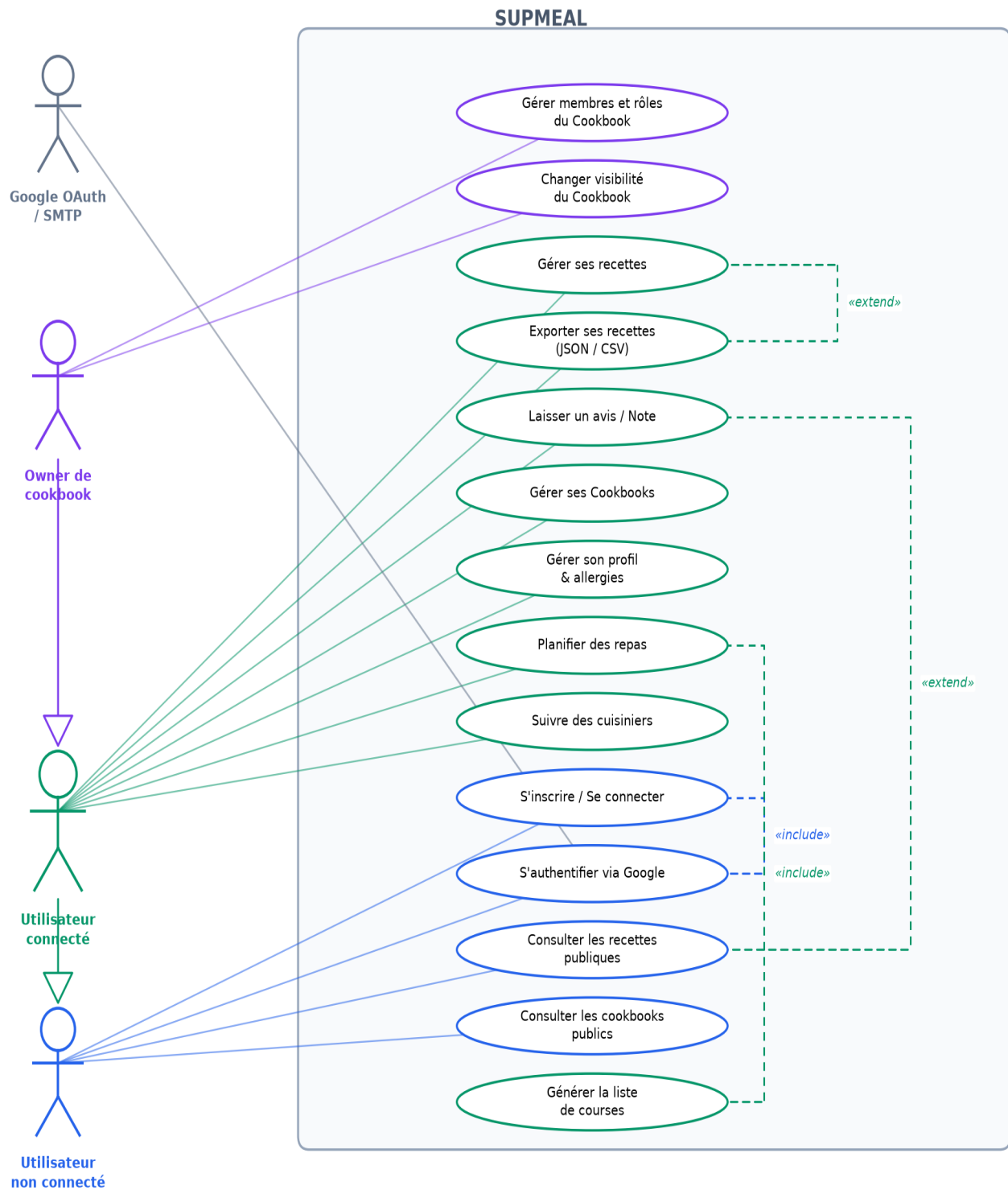
7. Diagramme de classes

Diagramme de classes — SUPMEAL



8. Diagramme de cas d'utilisation

Diagramme de cas d'utilisation — SUPMEAL



9. Sécurité

- **Mots de passe** : hachés avec bcrypt, jamais stockés en clair
- **Authentification** : JWT avec expiration 7 jours, validé à chaque requête protégée
- **Permissions** : système de rôles (OWNER / EDITOR / READER / COMMENTER) vérifié côté serveur à chaque appel
- **Uploads** : validation du type MIME (jpeg / png / webp uniquement) et limite de taille (5 Mo)
- **Variables sensibles** : aucun secret dans le code source, tout via variables d'environnement
- **OAuth2** : flux standard avec code d'autorisation, token jamais exposé côté client directement

10. API Documentation

L'API REST est documentée et testable interactivement via Swagger UI : <http://localhost:3000/docs>

Authentification

Auth			^
GET	/api/auth/google	Démarrer la connexion OAuth2 avec Google	⌵
GET	/api/auth/google/callback	Callback OAuth2 Google (usage interne, redirige vers le frontend avec un token)	🔗 ⌵
POST	/api/auth/register	Créer un compte utilisateur	⌵
POST	/api/auth/login	Se connecter	⌵
GET	/api/auth/me	Récupérer le profil de l'utilisateur connecté	🔒 ⌵
GET	/api/auth/verify-email	Vérifier l'email via token (lien reçu par email)	⌵
POST	/api/auth/forgot-password	Demander une réinitialisation de mot de passe	⌵
POST	/api/auth/reset-password	Réinitialiser le mot de passe avec un token	⌵

Recettes

Recipes			^
GET	/api/recipes	Lister mes recettes personnelles	🔒 ⌵
POST	/api/recipes	Créer une recette (personnelle, ou dans un cookbook si cookbookId fourni)	🔒 ⌵
GET	/api/recipes/{id}	Détail d'une recette	🔒 ⌵
PUT	/api/recipes/{id}	Modifier une recette	🔒 ⌵
DELETE	/api/recipes/{id}	Supprimer une recette	🔒 ⌵
PATCH	/api/recipes/{id}/favorite	Basculer le statut favori d'une recette	🔒 ⌵
PATCH	/api/recipes/{id}/public	Basculer la visibilité publique d'une recette perso	🔒 ⌵

Cookbooks

Cookbooks			^
GET	/api/cookbooks	Lister mes cookbooks	🔒 ⌵
POST	/api/cookbooks	Créer un cookbook	🔒 ⌵
GET	/api/cookbooks/{id}	Détail d'un cookbook (avec membres et recettes)	🔒 ⌵
DELETE	/api/cookbooks/{id}	Supprimer un cookbook (créateur uniquement)	🔒 ⌵
POST	/api/cookbooks/{id}/members	Inviter un membre par email (créateur uniquement)	🔒 ⌵
PATCH	/api/cookbooks/{id}/members/{userId}	Modifier le rôle d'un membre (créateur uniquement)	🔒 ⌵
DELETE	/api/cookbooks/{id}/members/{userId}	Retirer un membre (créateur uniquement)	🔒 ⌵
POST	/api/cookbooks/{id}/recipes	Ajouter une recette existante dans un cookbook (copie)	🔒 ⌵
GET	/api/cookbooks/public	Lister les cookbooks publics	🔒 ⌵
PATCH	/api/cookbooks/{id}/public	Basculer la visibilité publique d'un cookbook (OWNER uniquement)	🔒 ⌵
POST	/api/cookbooks/copy-recipe	Copier une recette publique dans ses recettes perso	📷 🔒 ⌵

Recherche, planification, profils

Users			^
PATCH	/api/users/me	Mettre à jour mon profil et mes préférences culinaires	🔒 ⌵
DELETE	/api/users/me	Supprimer mon compte définitivement	🔒 ⌵
PATCH	/api/users/me/password	Changer mon mot de passe	🔒 ⌵
Search			^
GET	/api/search	Rechercher et filtrer des recettes (perso + cookbooks accessibles)	🔒 ⌵

Profiles			^
GET	/api/profiles	Rechercher des cuisiniers par nom	🔒 ⌵
GET	/api/profiles/feed	Fil d'actualité des cuisiniers suivis	🔒 ⌵
GET	/api/profiles/{userId}	Profil public d'un cuisinier	🔒 ⌵
POST	/api/profiles/{userId}/follow	Suivre ou ne plus suivre un cuisinier	🔒 ⌵

MealPlan

GET	/api/mealplan	Récupérer le planning d'une semaine	🔒	⌵
POST	/api/mealplan	Ajouter une recette au planning	🔒	⌵
DELETE	/api/mealplan/{id}	Retirer une recette du planning	🔒	⌵
GET	/api/mealplan/shopping-list	Générer la liste de courses de la semaine	🔒	⌵

Import / Export et invitations

Export/Import

GET	/api/export	Exporter mes recettes (perso + cookbooks accessibles)	🔒	⌵
POST	/api/import	Importer des recettes (attribution automatique à l'utilisateur courant)	🔒	⌵

Invitations

GET	/api/invitations	Lister mes invitations en attente	🔒	⌵
GET	/api/invitations/count	Nombre d'invitations en attente	🔒	⌵
POST	/api/invitations/{invitationId}/respond	Accepter ou refuser une invitation	🔒	⌵
POST	/api/cookbooks/{id}/invite	Inviter un utilisateur dans un cookbook (crée une invitation à accepter)	🔒	⌵